SELF-LEARNING ACTIVITY 2
Case Study: Hospital Management System

1. Aim and Objectives

Aim: To study software design and architecture concepts by applying UML, SOLID principles, refactoring techniques, code-smell identification, and Domain-Driven Design to a Hospital Management System.

Objectives

- Understand the purpose of StarUML and common UML relationships.
- Design the main workflows and structure of a Hospital Management System using UML diagrams.
- Identify poor software design and improve it using the five SOLID principles.
- Recognize common code smells and apply suitable refactoring techniques.
- Identify DDD concepts such as entities, value objects, aggregates, repositories, services, and bounded contexts.
- Compare object-oriented design with structured design for the same hospital workflow.

2. Introduction and Case Study Overview

A Hospital Management System (HMS) is software used to manage patient registration, doctor details, appointments, consultations, prescriptions, billing, and payments. It reduces paperwork, keeps information organized, and helps hospital staff access the correct information.

Problem Statement

If a hospital manages all activities manually or through separate, disconnected files, staff may face duplicate records, appointment clashes, delayed billing, and difficulty finding patient history. The proposed HMS combines these activities in one organized system with clear responsibilities for each module.

Main Users / Actors

Actor	Responsibility
Patient	Registers, requests appointments, and receives prescriptions and bills.
Receptionist	Registers patients and schedules or updates appointments.
Doctor	Views appointment details, records consultation notes, and issues prescriptions.
Billing Staff	Creates bills and records payments.
Administrator	Manages user accounts, doctors, departments, and system settings.

Basic Workflow

Hospital workflow

Patient Registration
↓
Book Appointment
↓
Doctor Consultation
↓

Create Prescription (if required)

↓

Generate Bill → Record Payment

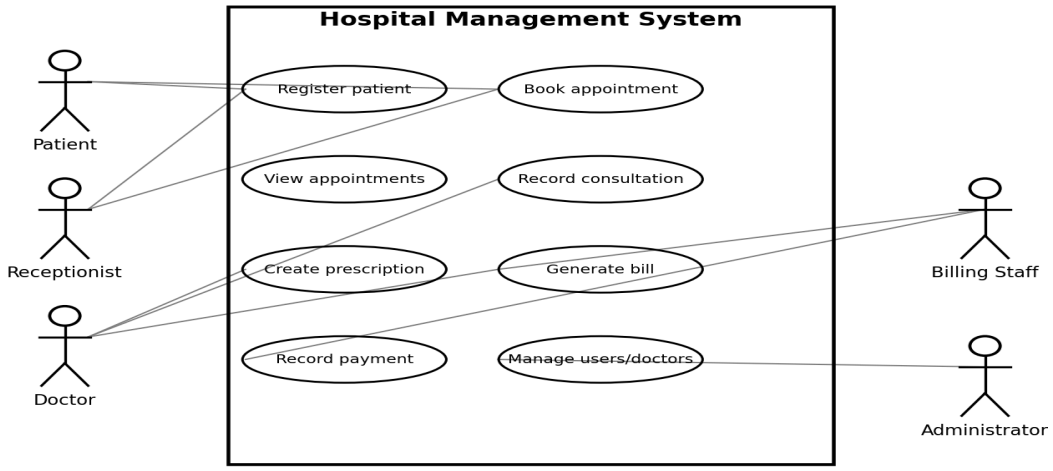
3. StarUML and UML Diagrams

StarUML is a modelling tool used to create UML (Unified Modeling Language) diagrams. UML diagrams help developers understand system users, classes, interactions, activities, and deployment before writing the complete code.

3.1 UML Relationships

Relationship	Meaning	Hospital example
Association	A general link between two classes; shown as a solid line.	Doctor treats Patient.
Inheritance	A child class inherits properties and methods from a parent; shown as a solid line with a hollow triangle pointing to the parent.	Doctor and Receptionist inherit from HospitalStaff.
Aggregation	A weak whole-part relationship; parts can exist independently; shown with a hollow diamond at the whole.	Department ◇— Doctor (a doctor can still exist if a department changes).
Composition	A strong whole-part relationship; the part's lifecycle depends on the whole; shown with a filled diamond at the whole.	Appointment ◆— ConsultationRecord (in this simplified model, the record belongs to its appointment).
Dependency	One element temporarily uses another; shown with a dashed arrow.	AppointmentService - - > NotificationService.

3.2 Use Case Diagram



Purpose: Shows what each user can do with the system.

Use case diagram — recreate in StarUML

Actors: Patient | Receptionist | Doctor | Billing Staff | Administrator

Patient -------- (Request Appointment)

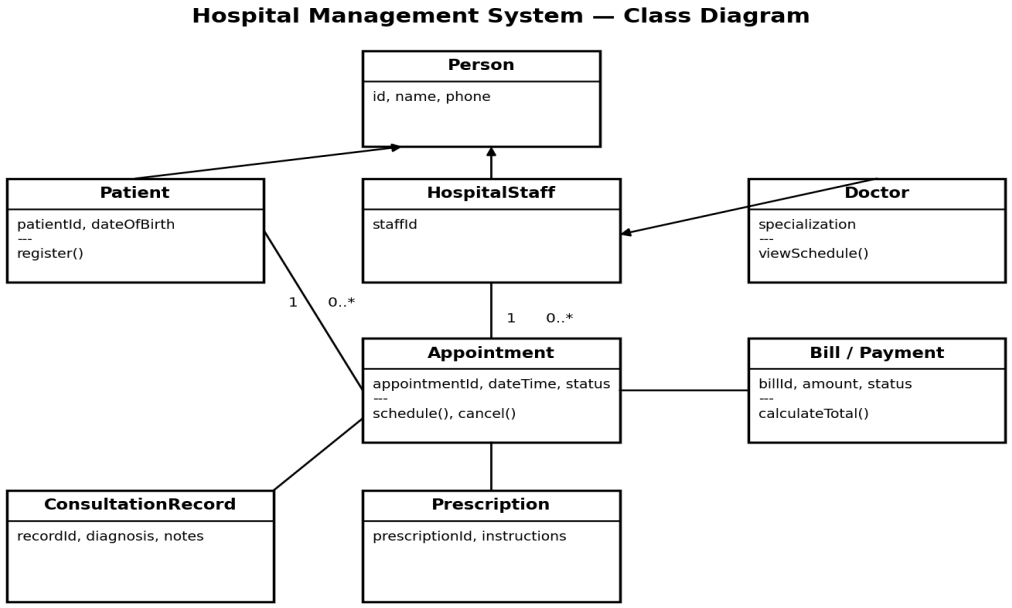
Receptionist --- (Register Patient) ---- (Schedule Appointment)

Doctor --------- (View Appointments) --- (Record Consultation)

Doctor --------- (Create Prescription)

Billing Staff -- (Generate Bill) -------- (Record Payment)
Administrator -- (Manage Users and Doctors)

3.3 Class Diagram

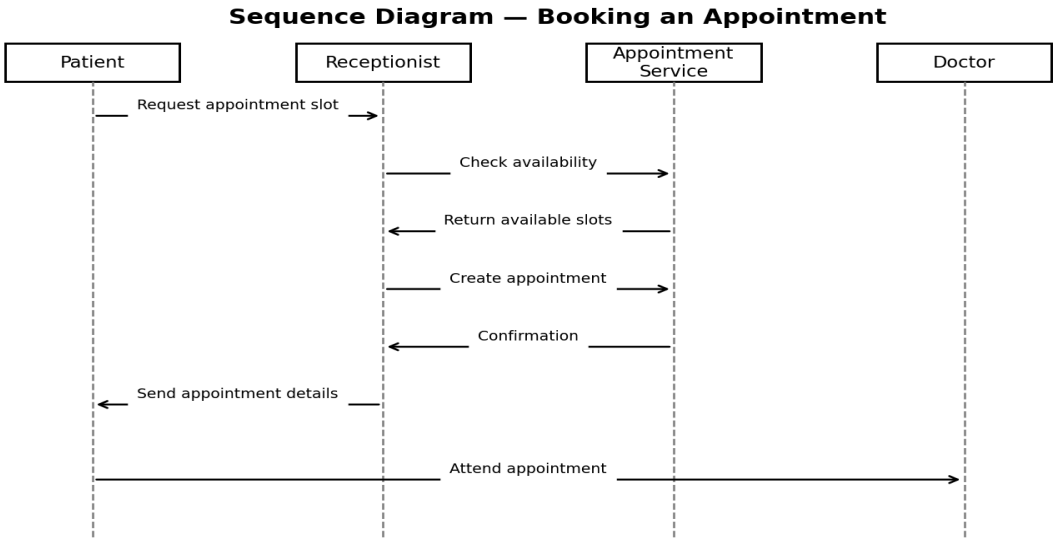


Purpose: Shows the main classes, their data, methods, and relationships.

Suggested attributes and operations: Patient: patientId, dateOfBirth; register(). Doctor: doctorId, specialization; viewSchedule(). Appointment: appointmentId, dateTime, status; schedule(), cancel(). Bill: billId, amount, status; calculateTotal().

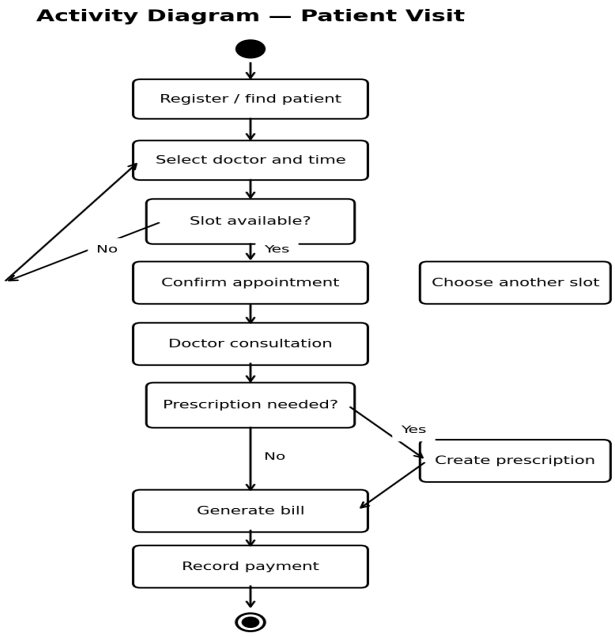
Multiplicity notes: one patient can have many appointments; each appointment is associated with one patient and one doctor. In this example, an appointment may have zero or one consultation record and zero or one bill.

3.4 Sequence Diagram



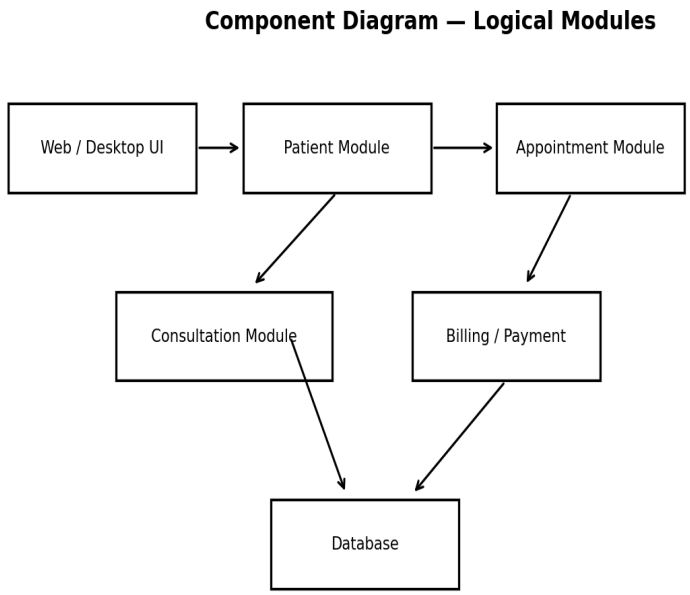
Purpose: Shows the order of messages exchanged between objects over time.

3.5 Activity Diagram

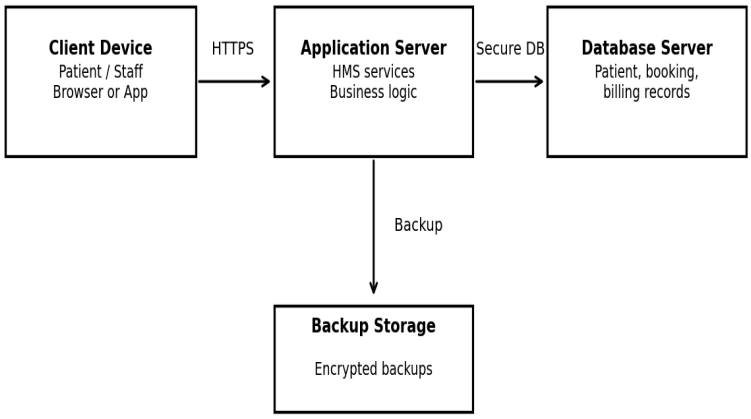


Purpose: Represents the flow of activities and decisions in a process.

3.6 Component and Deployment Diagrams



Deployment Diagram — Example Infrastructure



Component diagram purpose: Shows the main software modules and how they depend on each other.

4. Applying the SOLID Principles

SOLID is a set of five object-oriented design principles that make software easier to maintain, test, and extend. The examples below use a simplified Java-style design for the Hospital Management System.

4.1 S — Single Responsibility Principle (SRP)

Meaning: A class should have one main responsibility and one main reason to change.

Bad design:

```
class HospitalManager {
   void registerPatient() { /* registration */ }
   void bookAppointment() { /* scheduling */ }
   void generateBill() { /* billing */ }
   void sendReminder() { /* notifications */ }
}
```

Problem: One class handles unrelated tasks. A billing change could affect patient registration or reminders.

Refactored design:

```
class PatientService { void registerPatient() { } }
class AppointmentService { void bookAppointment() { } }
class BillingService { void generateBill() { } }
class NotificationService { void sendReminder() { } }
```

Applied principle: SRP. Each class has a clear responsibility.

4.2 O — Open/Closed Principle (OCP)

Meaning: Software entities should be open for extension but closed for repeated modification of existing, tested code.

Bad design:

```
class PaymentService {
    void pay(String method) {
        if (method.equals("Cash")) { /* cash payment */ }
        else if (method.equals("Card")) { /* card payment */ }
        else if (method.equals("UPI")) { /* UPI payment */ }
    }
}
```

Problem: Every new payment method requires changing the same class, which can introduce bugs.

Refactored design:

```
interface PaymentMethod {
    void pay(double amount);
}
class CashPayment implements PaymentMethod {
    public void pay(double amount) { /* process cash */ }
}
class UpiPayment implements PaymentMethod {
    public void pay(double amount) { /* process UPI */ }
}
class PaymentService {
    void process(PaymentMethod method, double amount) {
        method.pay(amount);
    }
}
```

Applied principle: OCP. New payment types can be added by implementing PaymentMethod.

4.3 L — Liskov Substitution Principle (LSP)

Meaning: An object of a child class should be usable wherever its parent type is expected without breaking correct behaviour.

Bad design: A general HospitalStaff class contains a prescribeMedicine() method, but a Receptionist subclass cannot prescribe medicine. The subclass would need to reject or leave the method unsupported.

Refactored design:

```
class HospitalStaff { void viewWorkSchedule() { } }
interface CanPrescribe {
    void prescribeMedicine();
}
class Doctor extends HospitalStaff implements CanPrescribe {
    public void prescribeMedicine() { /* create prescription */ }
}
class Receptionist extends HospitalStaff {
    // No prescription method required
}
```

Applied principle: LSP. Only staff types that genuinely support prescribing implement CanPrescribe.

4.4 I — Interface Segregation Principle (ISP)

Meaning: A class should not be forced to implement methods it does not need.

Bad design: One HospitalOperations interface requires registerPatient(), prescribeMedicine(), generateBill(), and manageUsers(). Receptionist, Doctor, and BillingStaff would all have irrelevant methods.

Refactored design:

```
interface PatientRegistration { void registerPatient(); }
interface PrescriptionWriter { void prescribeMedicine(); }
interface BillingOperations { void generateBill(); }
interface UserAdministration { void manageUsers(); }
```

Applied principle: ISP. Small, focused interfaces give each role only the operations it needs.

4.5 D — Dependency Inversion Principle (DIP)

Meaning: High-level business logic should depend on abstractions rather than directly depending on low-level implementation details.

Bad design:

```
class AppointmentService {
    private MySqlAppointmentRepository repo =
        new MySqlAppointmentRepository();
}
```

Problem: AppointmentService is tightly tied to one database implementation, making testing and database changes harder.

Refactored design:

```
interface AppointmentRepository {
    void save(Appointment appointment);
}
class AppointmentService {
    private AppointmentRepository repo;
    AppointmentService(AppointmentRepository repo) {
        this.repo = repo;
    }
    void book(Appointment appointment) {
        repo.save(appointment);
    }
}
```

Applied principle: DIP. The service receives an abstraction, so a database repository or test fake can be supplied.

Principle	Problem addressed	Result in HMS
SRP	One class does many unrelated tasks.	Separate patient, appointment, billing, and notification services.

OCP	New options require editing existing conditional logic.	Add a new payment implementation.
LSP	A child class cannot safely behave as its parent promises.	Separate general staff behaviour from doctor-only behaviour.
ISP	Large interfaces force irrelevant methods on classes.	Create small role-specific interfaces.
DIP	Business logic directly creates a concrete database class.	Depend on repository interfaces and inject implementations.

5. Refactoring and Code Smells

A code smell is a sign that code may be difficult to understand, test, or maintain. It does not always mean the program is incorrect. Refactoring improves the internal structure without changing the expected external behaviour.

5.1 Long Method

Smell: A method handles registration, slot checking, appointment creation, billing, and notifications in one long block.

Refactoring: Extract Method. Split it into registerPatient(), checkSlotAvailability(), createAppointment(), generateBill(), and sendConfirmation().

5.2 Large / God Class

Smell: A HospitalManager class contains patient, doctor, appointment, billing, inventory, and reporting logic.

Refactoring: Extract Class. Create PatientService, AppointmentService, BillingService, InventoryService, and ReportService.

5.3 Duplicate Code

Smell: The same patient contact-validation code is copied into registration and appointment forms.

Refactoring: Extract Method or introduce a shared validator, such as ContactValidator.validatePhone(). Keep business rules in an appropriate shared component.

5.4 Long Parameter List

Smell: createAppointment(patientId, doctorId, date, time, department, room, reason, phone) has too many parameters.

Refactoring: Introduce a parameter object such as AppointmentRequest, containing the related booking details.

```
class AppointmentRequest {
   String patientId;
   String doctorId;
   String dateTime;
   String reason;
```

```
}
appointmentService.book(request);
```

5.5 Tight Coupling

Smell: AppointmentService directly creates a concrete database connection and sends messages through a specific SMS provider.

Refactoring: Depend on interfaces such as AppointmentRepository and NotificationService, then inject the implementations.

Code smell	Why it is a problem	Refactoring technique
Long Method	Hard to read, test, and debug.	Extract Method
God Class	Too many responsibilities and dependencies.	Extract Class; apply SRP
Duplicate Code	Fixes must be repeated and copies may differ.	Extract Method / shared validator
Long Parameter List	Calls are hard to read and easy to mix up.	Introduce Parameter Object
Tight Coupling	Changing one component affects others.	Dependency Injection; depend on interfaces

6. Domain-Driven Design (DDD)

Domain-Driven Design is an approach that structures software around the real business domain and its rules. For an HMS, the domain includes patients, appointments, consultations, prescriptions, and billing.

6.1 DDD Building Blocks

Building block	Meaning	HMS example
Entity	Object with a stable identity that can change over time.	Patient(patientId), Doctor(doctorId), Appointment(appointmentId).
Value Object	Object defined by its values rather than a unique identity; ideally immutable.	Money(amount, currency), DateTimeRange, Address.
Aggregate	Group of related domain objects controlled through an aggregate root.	Appointment as root controlling its booking status and related booking rules.
Repository	Abstraction for loading and saving domain objects.	AppointmentRepository finds and saves appointments.
Domain Service	Business operation that does not naturally belong to one entity.	AppointmentAvailabilityService checks doctor and time-slot availability.
Bounded Context	A boundary within which a domain model and its terms have a specific meaning.	Scheduling, Clinical Records, and Billing contexts.

6.2 Example Aggregate and Business Rules

Appointment can be treated as an aggregate root. The application asks Appointment to confirm or cancel a booking rather than changing its status freely from unrelated parts of the system.

- An appointment must refer to an existing patient and doctor.
- A doctor should not have two confirmed appointments at the same time.
- A cancelled appointment should not be treated as a confirmed booking.
- A bill should record the amount due and the payment status.

6.3 Possible Bounded Contexts

Bounded context	Main concepts	Responsibility
Patient Management	Patient, ContactDetails	Registration and demographic details.
Scheduling	Appointment, DoctorSchedule, TimeSlot	Booking, rescheduling, cancellation, availability.
Clinical Records	Consultation, Diagnosis, Prescription	Consultation and clinical documentation; access must be restricted.
Billing	Bill, Payment, Money	Charges, invoices, payments, and payment status.
Identity and Access	User, Role, Permission	Authentication and authorization.

These contexts may be separate modules in a small project. They do not have to be separate microservices. The correct boundary depends on the system's size and operational needs.

7. Object-Oriented Design vs Structured Design

Both approaches can solve the same hospital workflow, but they organize the solution differently.

7.1 Object-Oriented Design (OOD)

OOD organizes the system around objects that combine data and behaviour. Classes such as Patient, Doctor, Appointment, Prescription, and Bill represent domain concepts.

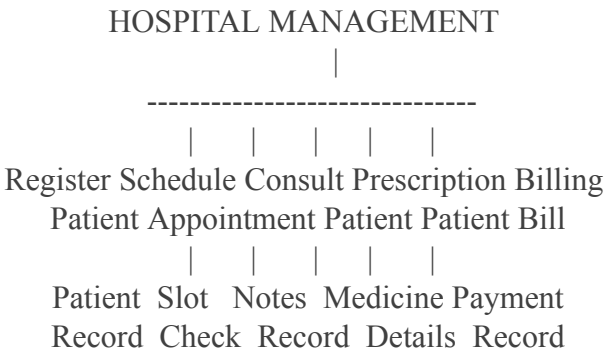
```
class Appointment {
   private String status;
   public void confirm() {
      if (!status.equals("CANCELLED")) {
         status = "CONFIRMED";
      }
   }
}
```

Here, the Appointment object controls a rule about its own status.

7.2 Structured Design

Structured design organizes the solution around functions or processes and the data passed between them. A top-down design may divide the work into registration, scheduling, consultation, prescription, and billing procedures.

Structured design — top-down decomposition



7.3 Comparison Table

Basis	Object-Oriented Design	Structured Design
Main focus	Objects and their responsibilities.	Functions, processes, and data flow.
Basic unit	Class / object.	Function / procedure / module.
Data handling	Data and related behaviour are grouped together.	Data is commonly passed between functions or stored in shared records.
Approach	Often models the domain using objects and relationships.	Often uses top-down functional decomposition and data-flow models.
Reuse	Inheritance, composition, interfaces, and object composition.	Function and module reuse.
Change impact	Well-designed classes can localize changes to one responsibility.	Changes to shared data structures can affect several functions.
HMS example	Appointment.confirm() controls appointment state.	confirmAppointment(appointmentRecord) updates the appointment record.
Best fit	Systems with rich domain rules and evolving entities.	Process-focused tasks with clear, stable procedures.

7.4 Which Is More Suitable for HMS?

Object-oriented design is a strong choice for a Hospital Management System because patients, appointments, prescriptions, and bills have identities, relationships, and business rules. Structured design is also useful for representing workflows and decomposing functions. In practice, a system can combine object-oriented domain models with structured process flows.

7.5 Case Study Workflow

Same workflow in both approaches

Register Patient → Book Appointment → Consult Doctor
→ Generate Prescription (if required) → Generate Bill

OOD: Patient / Appointment / Doctor / Prescription / Bill objects
Structured: registerPatient() / bookAppointment() / consultDoctor()
/ createPrescription() / generateBill()

8. Conclusion

Through this case study, I understood how software design techniques can be applied to a Hospital Management System. StarUML and UML diagrams help visualize users, classes, workflows, interactions, and deployment. SOLID principles divide responsibilities, make the design easier to extend, and reduce tight coupling. Refactoring improves code structure by removing common code smells without changing expected behaviour. DDD organizes the system around hospital concepts and business rules. Finally, object-oriented design focuses on objects and their responsibilities, while structured design focuses on processes and functions. Using these techniques together can make hospital software more understandable, maintainable, and reliable.

9. References

- StarUML official website and documentation: https://staruml.io/
- Martin, R. C., Clean Architecture: A Craftsman's Guide to Software Structure and Design, Prentice Hall.
- Fowler, M., Refactoring: Improving the Design of Existing Code, 2nd Edition, Addison-Wesley.
- Evans, E., Domain-Driven Design: Tackling Complexity in the Heart of Software, Addison-Wesley.
- OMG UML specifications and overview: https://www.omg.org/spec/UML/